Dossier d’architecture technique SESAM

Architecture web en n-tiers conteneurisée

Table des matières

1 Objet et périmètre 2
2 Architecture générale et échanges 3
3 Déploiement sur Docker 4
4 Interfaces et protocoles utilisés 5
5 Justification des choix techniques 6
6 Versions du socle technique 7
Modules backend 7
7 Environnements et exploitation 8
Configuration 8
Procédure de déploiement de développement 8
8 Hébergement sur Internet 9
Authentification et journaux 9
Exécution des conteneurs 9

1 Objet et périmètre

SESAM repose sur une architecture en n-tiers : une interface React, une application backend Spring Boot unique et une base PostgreSQL. L'interface couvre notamment la gestion des bâtiments, des salles, des matériels et des comptes. Les échanges métier utilisent une API REST et des données JSON.

Ce document décrit les responsabilités des composants, leurs échanges, le déploiement Docker de développement et les dispositions d'exploitation. Il présente séparément les adaptations nécessaires à l'hébergement cible C1NP. Deux emplacements sont réservés aux schémas d'architecture générale et de déploiement.

Les références CCT renvoient au Cadre de cohérence technique du ministère des Armées 2023–2024, version 3.5, édition 2. Les numéros de page correspondent à cette édition. L'applicabilité des versions logicielles et des exigences devra être confirmée avec la gouvernance technique pour l'hébergement cible.

2 Architecture générale et échanges

Le frontend assure la présentation et les interactions avec l'utilisateur. Le backend expose les services métier, contrôle les droits, valide les données et gère leur persistance. PostgreSQL conserve les données relationnelles ; les accès applicatifs à la base passent exclusivement par le backend.

Le serveur de l'interface et l'application React ont des rôles distincts. En développement, Node.js exécute Vite pour distribuer les fichiers de l'interface. Après leur chargement, le navigateur envoie les appels API au backend selon l'URL configurée dans le frontend. Le conteneur front ne sert pas de relais à ces appels.

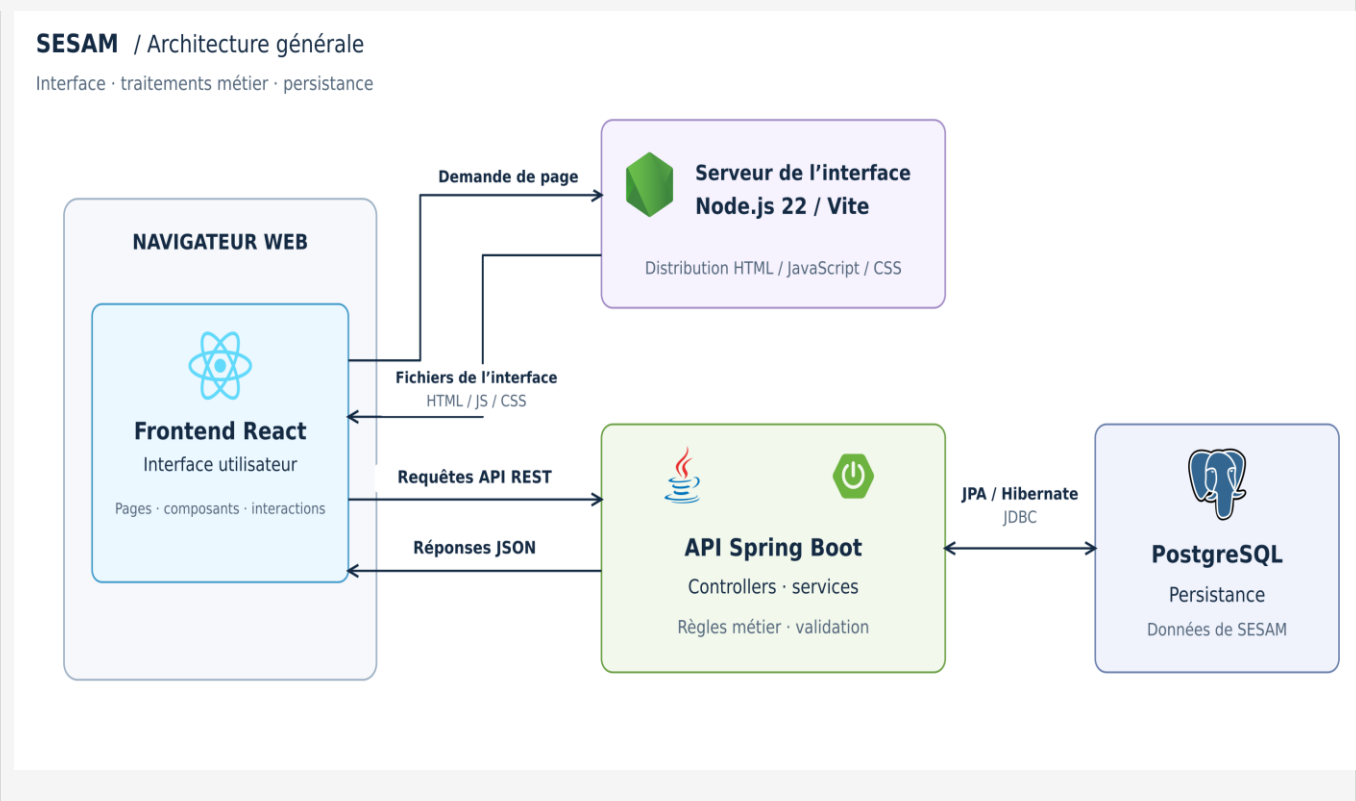


Figure 1 Architecture générale et échanges

Échange	Contenu et responsabilité
Navigateur et serveur de l'interface Node.js / Vite	Chargement de la page et des ressources nécessaires à l'interface React.
Application React et API Spring Boot	Fetch envoie les requêtes métier et reçoit les réponses JSON. Les contrôleurs Spring traitent les requêtes côté backend.
Backend et PostgreSQL	Spring Data JPA et Hibernate assurent la persistance. Le pilote PostgreSQL établit la connexion JDBC.

Spring Security protège les routes de l'API et contrôle les autorisations. L'authentification applicative utilise actuellement des JWT. L'intégration de l'authentification cible MindefConnect Internet est traitée dans la section 8.

3 Déploiement sur Docker

Le déploiement décrit correspond à l'environnement de production. Un hôte Docker, installé sur une machine physique ou une machine virtuelle, exécute trois conteneurs : front, backend et db. Docker Compose crée le réseau partagé de ces services.

Conteneur front. L'image de base est node:22-alpine. Le conteneur installe les dépendances avec npm ci puis lance Vite avec npm run dev -- --host 0.0.0.0. Le port 8081 de l'hôte, configurable par FRONT_PORT, donne accès au port 5173 du conteneur. Node.js exécute les outils du frontend ; Vite distribue l'interface en mode développement.

Conteneur backend. L'image de base maven:3.9.9-eclipse-temurin-21 fournit Maven et le JDK Java 21. La commande mvn spring-boot:run lance Spring Boot ; entr surveille les sources pour relancer l'application lors des modifications. Le port 8080 du conteneur est publié sur le port 8080 de l'hôte. Le backend applique les règles métier, l'authentification et les droits d'accès, puis accède aux données via JPA/Hibernate et JDBC.

Conteneur db. L'image postgres:17-alpine exécute PostgreSQL. Le backend la contacte par le nom de service db sur le port 5432 du réseau Docker. Aucun port de ce service n'est publié sur l'hôte.

Volume	Usage
postgres_data	Conservation des données PostgreSQL.
maven_cache	Conservation du cache des dépendances Maven.
front_node_modules	Conservation des dépendances Node.js du frontend.

Les sources ./Front et ./Backend sont également montées dans /workspace de leurs conteneurs respectifs. Ces deux montages de dossiers sont distincts des trois volumes Docker. Les volumes de dépendances facilitent le développement ; seul postgres_data contient les données métier.

Les paramètres sont fournis par variables d'environnement. Les valeurs par défaut du Compose servent au développement et doivent être remplacées pour les autres environnements. Le backend attend le contrôle de santé positif de db ; front dépend du démarrage de backend, sans contrôle de disponibilité applicative associé.

La production nécessite des images et des commandes d'exécution adaptées, la suppression des montages de sources et la définition des accès réseau, des secrets et de l'observabilité avec l'hébergeur.

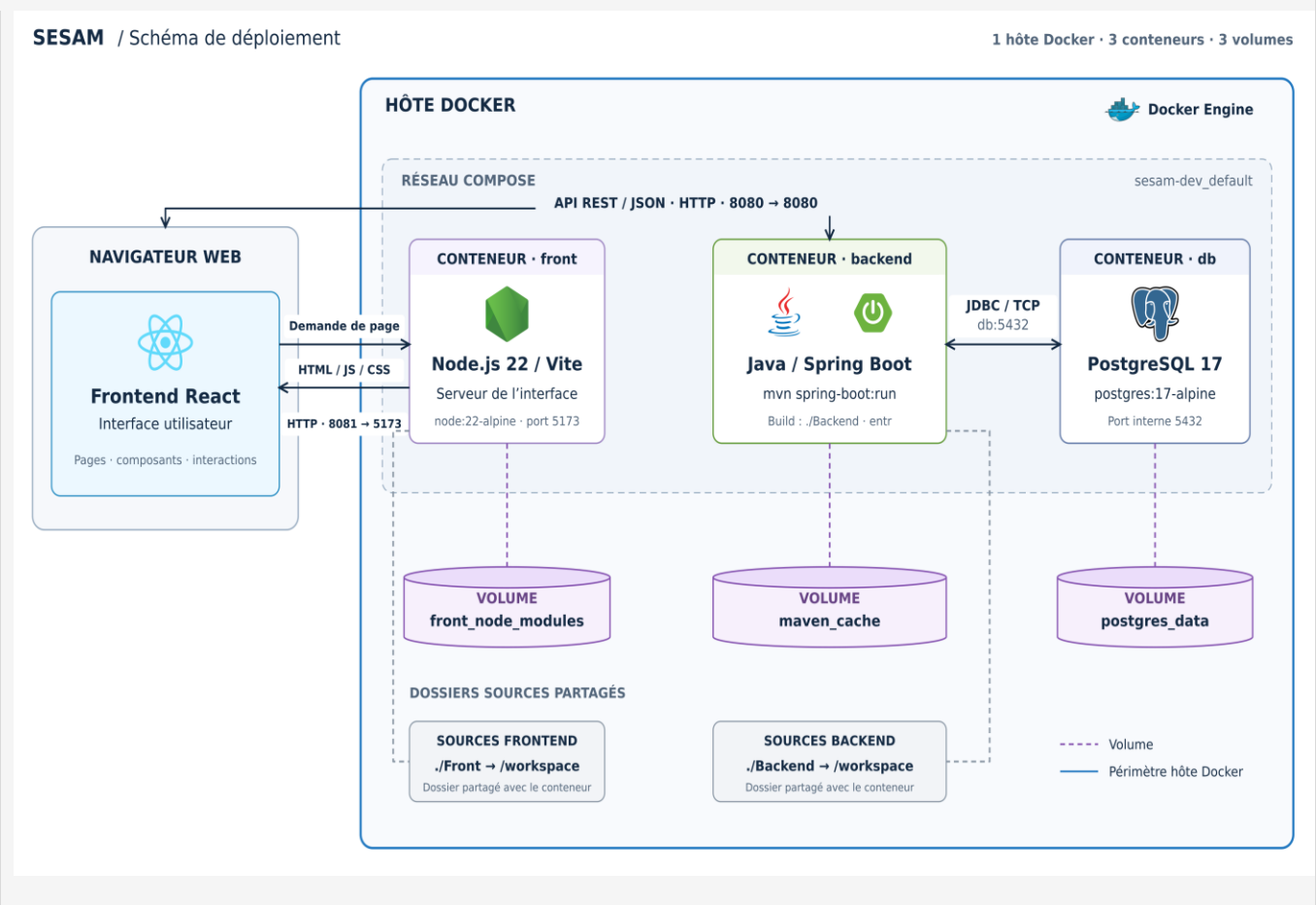


Figure 2 Déploiement de développement sur un hôte Docker

4 Interfaces et protocoles utilisés

En développement, le navigateur accède séparément au serveur de l'interface et à l'API du backend. Le frontend utilise l'URL définie par VITE_API_URL ; le backend utilise FRONT_ORIGIN pour l'origine frontend autorisée. Les ports publiés doivent être distingués du port PostgreSQL accessible entre services Docker.

Échange	Version ou service	Ports	Protocole
Navigateur vers serveur de l'interface	Node.js 22 / Vite 8	Hôte 8081 → front 5173 par défaut	HTTP
React vers API métier	Spring Boot 4.1.1 / Java 21	Hôte 8080 → backend 8080	HTTP / REST / JSON
Backend vers base de données	PostgreSQL 17 / db	db:5432 sur le réseau Docker	JDBC / protocole PostgreSQL sur TCP

L'URL d'API par défaut est http://localhost:8080 et l'origine frontend est http://localhost:8081. Ces adresses sont adaptées à un navigateur utilisé sur l'hôte Docker. Pour un accès depuis une autre machine, les URL et l'origine autorisée doivent être configurées avec une adresse joignable depuis ce navigateur.

5 Justification des choix techniques

Les technologies retenues correspondent aux compétences maîtrisées par l'équipe de développement. Ce choix réduit le temps de prise en main, facilite la maintenance et limite les risques d'erreurs pendant le développement. Il rejoint le principe de rationalisation des techniques et des compétences du CCT (§ 3.2.1.1, p. 60).

La séparation en n-tiers distingue l'interface, les règles métier et la persistance. Elle permet de faire évoluer chaque partie tout en conservant des interfaces explicites. L'interface web évite une installation applicative spécifique sur les postes, en cohérence avec le principe de client léger du CCT (§ 3.2.1.1, p. 60).

React et TypeScript. React permet de composer des vues réutilisables adaptées aux écrans de gestion de SESAM. TypeScript détecte certaines incohérences de types pendant le développement. Le CCT recommande les applications SPA (RT_DEV_10, § 7.3.5, p. 202), référence React comme recommandé (p. 204) et recommande TypeScript (RT_DEV_14, p. 203).

Spring Boot et Java 21. Spring Boot structure l'API REST et réunit les fonctions de validation, de sécurité et d'accès aux données. Le CCT recommande Spring et décrit les apports de Spring Boot (§ 7.3.2, pp. 200–201).

Spring Security et JWT. Spring Security centralise les contrôles d'accès du backend. JJWT permet de créer et de vérifier les jetons utilisés par l'authentification applicative actuelle.

PostgreSQL. Le modèle relationnel convient aux données structurées de SESAM et aux liens entre bâtiments, salles, matériels et utilisateurs. Il permet d'exprimer des contraintes et de conserver des données transactionnelles. Le CCT privilégie PostgreSQL pour les nouveaux développements et les migrations (§ 3.2.5.3.1, pp. 86–87).

Node.js et Vite. Node.js fournit le runtime des outils frontend ; Vite sert l'interface pendant le développement et produit les fichiers du build. Le CCT restreint Node.js aux versions paires LTS soutenues (§ 3.2.2.2.4, p. 72). Le maintien de cette condition doit être vérifié à chaque évolution. Le choix de Vite répond aux besoins d'outillage du projet.

Bootstrap 5. Les composants et styles réutilisables accélèrent la réalisation des écrans et rendent leur présentation homogène. Bootstrap 5 est référencé comme recommandé dans l'annexe des produits du CCT (§ 8.2.1, p. 237).

Docker et Logback. Docker Compose rend l'environnement de développement reproductible et isole ses services. Le choix du runtime de production suit les contraintes OCI du CCT (§ 5.1.3, pp. 160–161). Logback produit les journaux du backend pour leur collecte ultérieure.

6 Versions du socle technique

Les versions ci-dessous sont déclarées dans les fichiers du projet. Pour npm, ^ et ~ définissent des plages de versions ; package-lock.json fixe les versions installées par npm ci. Les tags d'images Docker peuvent évoluer : les versions exactes et les empreintes des images devront être tracées pour chaque livraison.

Technologie	Version déclarée	Fonction
React et React DOM	^19.2.8	Interface et rendu
TypeScript	~6.0.2	Typage statique
Vite	^8.3.0	Serveur de développement et build
React Router	^7.18.4	Navigation entre les vues
Bootstrap	^5.3.8	Composants et styles
Bootstrap Icons	^1.13.1	Icônes
ESLint	^10.10.0	Analyse statique
TypeScript ESLint	^8.69.0	Règles TypeScript
Java	21 / Eclipse Temurin JDK	Compilation et exécution en développement
Maven	3.9.9	Construction et lancement du backend
Spring Boot	4.1.1	Socle backend
JJWT	0.13.0	Gestion des jetons JWT
PostgreSQL	17 / postgres:17-alpine	Persistance relationnelle
Node.js	22 / node:22-alpine	Exécution des outils frontend
Logback	Gérée par les dépendances Spring Boot	Production des journaux du backend
SpringDoc OpenAPI	2.8.14	Génération de la documentation OpenAPI et consultation interactive avec le module springdoc-openapi-starter-webmvc-scalar

Modules backend

Spring Web, Spring Data JPA, Spring Validation, Spring Security et Spring Boot Actuator composent le socle backend. Le pilote PostgreSQL, Lombok et RSQL le complètent. Springdoc OpenAPI 2.8.14 génère la documentation de l'API ; le module springdoc-openapi-starter-webmvc-scalar fournit l'interface de consultation interactive Scalar. Les dépendances jakarta.json-api et Parsson assurent le traitement des documents JSON utilisés pour les opérations JSON Patch. Les versions transitives, dont celle de Logback, doivent figurer dans l'inventaire issu de l'arbre Maven et dans la nomenclature logicielle de livraison.

7 Environnements et exploitation

Environnement	Organisation du code	Usage
Développement	Branches dédiées aux fonctionnalités	Développement et vérifications locales.
Production	main	Version destinée à être livrée aux utilisateurs.

Chaque fonctionnalité est développée dans une branche dédiée. Après revue et validation, les changements sont intégrés dans main. Une livraison doit être associée à un commit et à des images identifiés ; la branche main ne déclenche pas, à elle seule, un déploiement en production.

Configuration

Les variables de connexion PostgreSQL, le secret JWT, l'URL de l'API et l'origine autorisée sont configurables. Le Compose actuel contient des valeurs de repli de développement. Les secrets réels doivent être fournis par un mécanisme externe adapté à l'environnement, sans intégration dans le code ou les images. Les variables VITE_ exposées au frontend sont publiques et ne doivent contenir aucun secret.

Procédure de déploiement de développement

1. Identifier le commit à installer et construire les images front et backend avec une référence traçable.
2. Préparer les variables d'environnement, les deux dossiers sources partagés et les trois volumes Docker.
3. Sauvegarder la base existante avant une évolution du schéma. La configuration actuelle utilise Hibernate ddl-auto: update ; des migrations versionnées doivent être définies pour la production.
4. Démarrer les services et contrôler la base, l'API et l'interface. Vérifier également les URL configurées et l'origine autorisée.
5. Tester la connexion utilisateur et une opération métier complète avant de valider l'installation.

8 Hébergement sur Internet

Cette section présente les dispositions prévues pour l'hébergement cible sur Internet. Elles sont distinctes de la configuration Docker de développement décrite dans les sections précédentes.

Conformément au point 6.1.6 du Cadre de cohérence technique, relatif à l'hébergement sur Internet, l'hébergement de l'application est prévu sur le réseau C1NP. Ce déploiement implique de disposer d'une pile logicielle à jour et maintenue en condition opérationnelle et de sécurité (MCO/MCS), conformément aux conditions d'homologation déjà applicables aux hébergements actuels (§ 6.1.6.2, p. 184).

Authentification et journaux

Le projet est actuellement en cours d'homologation et l'intégration de MindefConnect Internet reste à réaliser. Pour les besoins de démonstration, le scénario d'authentification sera présenté à l'aide d'une page de connexion standard. Ce dispositif provisoire ne se substitue pas à l'authentification MindefConnect Internet exigée pour l'hébergement cible (§ 6.1.6.2, p. 184).

Un puits de logs sera utilisé afin de centraliser les journaux d'événements applicatifs et de faciliter leur consultation et leur analyse. Logback assure la génération des journaux du backend Spring Boot. Pour C1NP, ces journaux devront être exportés vers le puits fourni par le socle ; les modalités de collecte, de transmission et de conservation seront définies avec l'hébergeur (§ 6.1.6.2, p. 184).

Exécution des conteneurs

Docker reste l'outil du déploiement de développement. Pour l'hébergement Internet, la règle CCT_R6 impose le respect des standards OCI et cite notamment containerd, cri-o et runc comme runtimes envisageables, tandis que Docker est refusé dans ce contexte (§ 5.1.3, p. 161). Le runtime et le mode de déploiement devront donc être arrêtés avec l'hébergeur. L'architecture et la pile logicielle seront soumises au SC²A avec les Dockerfile ou équivalents et les nomenclatures logicielles SBOM, qui recensent les composants et leurs versions.